Python Library

```
import warnings
warnings.filterwarnings("ignore")

import re
import os
import pickle
import numpy                    as np
import pandas                   as pd
import seaborn                  as sns
import missingno                as msno
import matplotlib.pyplot        as plt
from sklearn.feature_selection import RFE
from sklearn.preprocessing     import LabelEncoder
from sklearn.model_selection   import train_test_split
from sklearn.ensemble          import RandomForestRegressor
from sklearn.tree              import DecisionTreeClassifier
from sklearn.ensemble          import GradientBoostingRegressor
from sklearn.metrics           import r2_score
from sklearn.metrics           import mean_squared_error
from sklearn.metrics           import mean_absolute_error
from sklearn.metrics           import mean_absolute_percentage_error
```

Global Parameter and Definition

```
TEST_SIZE    = 0.20
RANDOM_STATE = 331
```

Global Function

```
# Function to plot header
def plot_sub_header(text, long=False):

    if long:
        HEIGHT = 0.5 * 2
    else:
        HEIGHT = 0.5

    # Plot sub-header
    plt.figure(figsize=(16, HEIGHT), dpi=120)
    ax = plt.gca()
    plt.xticks([])
    plt.yticks([])
    ax.set_facecolor("navy")
    plt.text(0.5, 0.5, text, color="yellow", fontsize=16,
fontweight="heavy", ha="center", va="center")
    plt.show()
```

```
def plot_add_text(axis, x, y):
    for i in range(len(x)):
        axis.text(i, y[i], str(round(float(y[i]), 2)) + "%",
horizontalalignment="center", verticalalignment="center")
```

Load Dataset

```
# Load house price dataset
df = pd.read_csv("data_kaggle.csv")

# Drop missing values
df = df.dropna()

# Plot header
plot_sub_header("Dataset Exploration")

# Visualize the top-10 rows of the processed CSV
df.head(10)
```

Dataset Exploration

```
                                   Location         Price Rooms
Bathrooms  \
0                        KLCC, Kuala Lumpur  RM 1,250,000   2+1
3.0
2                     Dutamas, Kuala Lumpur  RM 1,030,000     3
4.0
4                 Bukit Jalil, Kuala Lumpur    RM 900,000   4+1
3.0
5         Taman Tun Dr Ismail, Kuala Lumpur  RM 5,350,000   4+2
5.0
7         Taman Tun Dr Ismail, Kuala Lumpur  RM 2,600,000     5
4.0
8         Taman Tun Dr Ismail, Kuala Lumpur  RM 1,950,000   4+1
4.0
9                Sri Petaling, Kuala Lumpur    RM 385,000     3
2.0
13        Taman Tun Dr Ismail, Kuala Lumpur  RM 4,580,000   6+1
5.0
16  Bukit Tunku (Kenny Hills), Kuala Lumpur  RM 9,000,000   6+1
7.0
18                 Mont Kiara, Kuala Lumpur  RM 1,780,000   4+1
4.0

    Car Parks                      Property Type
Size  \
0         2.0                 Serviced Residence   Built-up : 1,335
sq. ft.
2         2.0               Condominium (Corner)   Built-up : 1,875
```

```
sq. ft.
4          2.0                Condominium (Corner)   Built-up : 1,513
sq. ft.
5          4.0                           Bungalow   Land area : 7200
sq. ft.
7          4.0                 Semi-detached House   Land area : 3600
sq. ft.
8          3.0  2-sty Terrace/Link House (EndLot)  Land area : 25x75
sq. ft.
9          1.0            Apartment (Intermediate)     Built-up : 904
sq. ft.
13         5.0             Bungalow (Intermediate)   Land area : 6000
sq. ft.
16         4.0                  Bungalow (Corner)   Land area : 8500
sq. ft.
18         2.0                Condominium (Corner)   Built-up : 1,830
sq. ft.

          Furnishing
0    Fully Furnished
2   Partly Furnished
4   Partly Furnished
5   Partly Furnished
7   Partly Furnished
8   Partly Furnished
9   Partly Furnished
13  Partly Furnished
16  Partly Furnished
18  Partly Furnished
```

```python
# Plot header
plot_sub_header("Dataset Statistical Data")

# Describe the dataset's attribute
df.describe()
```

Dataset Statistical Data

```
          Bathrooms     Car Parks
count  31899.000000  31899.000000
mean       2.936236      2.013449
std        1.532726      1.308361
min        1.000000      1.000000
25%        2.000000      1.000000
50%        2.000000      2.000000
75%        4.000000      2.000000
max       20.000000     30.000000
```

```python
# Plot header
plot_sub_header("Dataset Column Data")

# Visualize dataset's columns
df.columns
```

Dataset Column Data

```
Index(['Location', 'Price', 'Rooms', 'Bathrooms', 'Car Parks',
'Property Type',
       'Size', 'Furnishing'],
      dtype='object')
```

```python
# Plot header
plot_sub_header("Dataset Data Type")

# Visualize dataset's data types
df.dtypes
```

Dataset Data Type

```
Location          object
Price             object
Rooms             object
Bathrooms        float64
Car Parks        float64
Property Type     object
Size              object
Furnishing        object
dtype: object
```

Dataset Pre-processing

```python
# Process price column to remove currency sign
def col_price_remove_rm(price):
    return (price.replace("RM ", "").replace(",", ""))

# Process rooms column to standardize entries
def col_rooms_remove_unknown(rooms):
    return (rooms.replace(" Above", "").replace("Studio", "1"))

# Process rooms column to combine room numbers
def col_rooms_process_symbol(rooms):
    if "+" in str(rooms):
        try:
            rooms = int(rooms.split("+")[0].replace(" ", "")) +
int(rooms.split("+")[1].replace(" ", ""))
        except:
```

```
            pass
    rooms = str(rooms).replace("+", "")

    return int(rooms)

# Process property type column to remove parenthesis details
def col_property_type_group(property_type):
    property_type = re.sub("\(.*?\)", "", property_type).strip()

    return property_type

# Process unit column to replace measurement unit
def col_size_remove_unit(size):
    return (size.replace(" sq. ft.", "").replace(" sq. m.",
"").replace(",", ""))

# Process size column to standardize and combine measurement unit
def col_size_process_symbol(size):
    if "Land area" in str(size):
        new_size = size.replace("Land area : ", "")
        new_size = new_size.lower().replace(" ", "")
        if "x" in str(new_size):
            try:
                new_size = int(new_size.split("x")[0].replace(" ",
"")) * int(new_size.split("x")[1].replace(" ", ""))
            except:
                pass
        elif "*" in str(new_size):
            try:
```

```
                new_size = int(new_size.split("*")[0].replace(" ",
"")) * int(new_size.split("*")[1].replace(" ", ""))
            except:
                pass
        return new_size
    else:
        return size.replace("Built-up : ", "").replace(",", "")

# Process size column to remove unwanted entries
def col_size_process_symbol_unknown(size):
    try:
        size = int(size)
        if size == 0:
            return np.nan
        else:
            return size
    except:
        return np.nan

# Process size column by grouping
def col_size_grouping(size):
```

```
    try:
        size = int(size)
        if size < 500:
            size = "<500"
        elif size < 1000:
            size = "<1000"
        elif size < 5000:
            size = "<5000"
        elif size < 10000:
            size = "<10000"
        elif size < 20000:
            size = "<20000"
        else:
            size = ">=20000"
        return size
    except:
        pass

# Process furnishing column to remove unwanted entries
def col_furnishing_remove_unknown(furnishing):
    if "unknown" in str(furnishing).lower():
        return np.nan
    else:
        return furnishing
```

```
df["Price"          ] = df["Price"        ].apply(col_price_remove_rm)
df["Rooms"          ] =
df["Rooms"         ].apply(col_rooms_remove_unknown)
df["Rooms"          ] =
df["Rooms"         ].apply(col_rooms_process_symbol)
df["Property Type" ] = df["Property
Type"].apply(col_property_type_group)
df["Size"           ] = df["Size"         ].apply(col_size_remove_unit)
df["Size"           ] =
df["Size"          ].apply(col_size_process_symbol)
df["Size"           ] =
df["Size"          ].apply(col_size_process_symbol_unknown)
df["Size Histogram"] = df["Size"         ].apply(col_size_grouping)
df["Furnishing"     ] =
df["Furnishing"    ].apply(col_furnishing_remove_unknown)

df['Price'] = df['Price'].str.replace('RM', '').str.replace(',',
'').astype(float)

# Plot header
plot_sub_header("Pre-processed Dataset")

df = df.dropna()

df["Bathrooms"     ] = df["Bathrooms"].astype("int64")
```

```
df["Car Parks"    ] = df["Car Parks"].astype("int64")
df["Size"         ] = df["Size"     ].astype("int64")
df
```

Pre-processed Dataset

```
                                Location      Price  Rooms  Bathrooms
\
0                     KLCC, Kuala Lumpur  1250000.0      3          3

2                  Dutamas, Kuala Lumpur  1030000.0      3          4

4              Bukit Jalil, Kuala Lumpur   900000.0      5          3

5      Taman Tun Dr Ismail, Kuala Lumpur  5350000.0      6          5

7      Taman Tun Dr Ismail, Kuala Lumpur  2600000.0      5          4

...                                  ...        ...    ...        ...

53862              Seputeh, Kuala Lumpur   750000.0      3          2

53868           KL Sentral, Kuala Lumpur  1400000.0      4          3

53869          KL Eco City, Kuala Lumpur   880000.0      1          1

53876         Sri Hartamas, Kuala Lumpur  2700000.0      6          6

53882              Setapak, Kuala Lumpur   540000.0      4          2
```

```
       Car Parks        Property Type  Size        Furnishing Size
Histogram
0              2   Serviced Residence  1335   Fully Furnished
<5000
2              2          Condominium  1875  Partly Furnished
<5000
4              2          Condominium  1513  Partly Furnished
<5000
5              4             Bungalow  7200  Partly Furnished
<10000
7              4  Semi-detached House  3600  Partly Furnished
<5000
...          ...                  ...   ...               ...
...
53862          1          Condominium   915  Partly Furnished
<1000
53868          2          Condominium  1544   Fully Furnished
<5000
```

```
53869          1          Condominium   650  Partly Furnished
<1000
53876          3          Condominium  3973  Partly Furnished
<5000
53882          3          Condominium  1313  Partly Furnished
<5000

[31485 rows x 9 columns]
```

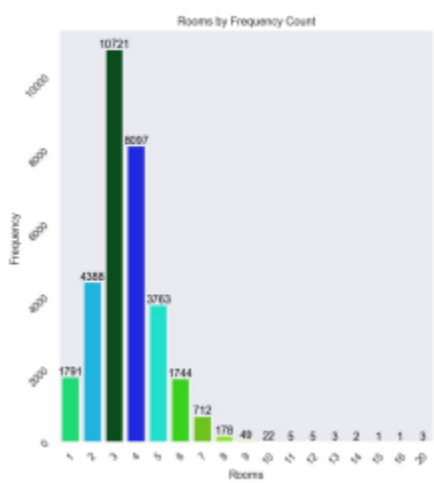
Exploratory Data Analysis (EDA)

```
# EDA Plot
EDA_COL = ["Rooms", "Bathrooms", "Car Parks", "Size Histogram"]
EDA_PLT = "gist_ncar"

for idx, col in enumerate(EDA_COL):

    # Plot header
    plot_sub_header("{} by Frequency Count".format(col))

    fig, ax = plt.subplots(1, 2, figsize=(16, 8), dpi=120)

    x    = df[col].value_counts().index
    y    = df[col].value_counts().values.tolist()
    data = df.groupby(col).size()

    sns.set(
        style       = "dark",
        color_codes = True
    )

    rank = data.argsort().argsort()
```

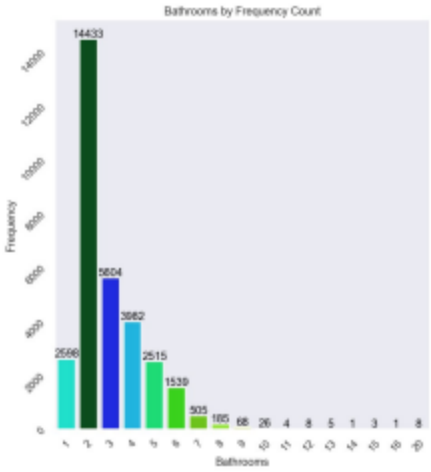
```
    sns.barplot(
        x       = x,
        y       = y,
        palette = np.array(sns.color_palette(EDA_PLT, len(data))[::-
1])[rank],
        ax      = ax[0]
    ).set_title(
        "{} by Frequency Count".format(col)
    )

    ax[0].tick_params(
        labelrotation = 45
    )

    ax[0].set(
        xlabel = "{}".format(col),
        ylabel = "Frequency"
```

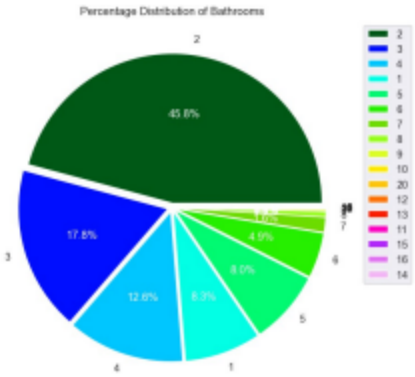
```
    )

    for p in ax[0].patches:
        ax[0].annotate(
            "{:.0f}".format(p.get_height()),
            (p.get_x()+0.40, p.get_height()),
            ha    = "center",
            va    = "bottom",
            color = "black"
        )

    _, _, autotexts= ax[1].pie(
        y,
        labels  = x,
        colors  = sns.color_palette(EDA_PLT, len(data)),
        autopct = "%1.1f%%",
        explode = [0.03 for i in df[col].value_counts().index]
    )

    plt.title("Percentage Distribution of {}".format(col))

    for autotext in autotexts:
        autotext.set_color("white")

    plt.legend(bbox_to_anchor=(1, 1))
    plt.show()
    print("\n\n")
```

Rooms by Frequency Count

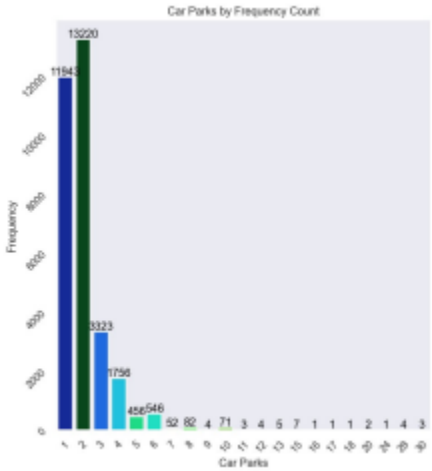


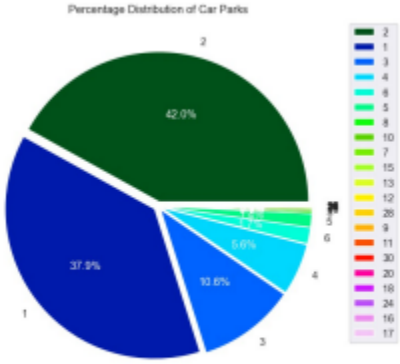
Bathrooms by Frequency Count

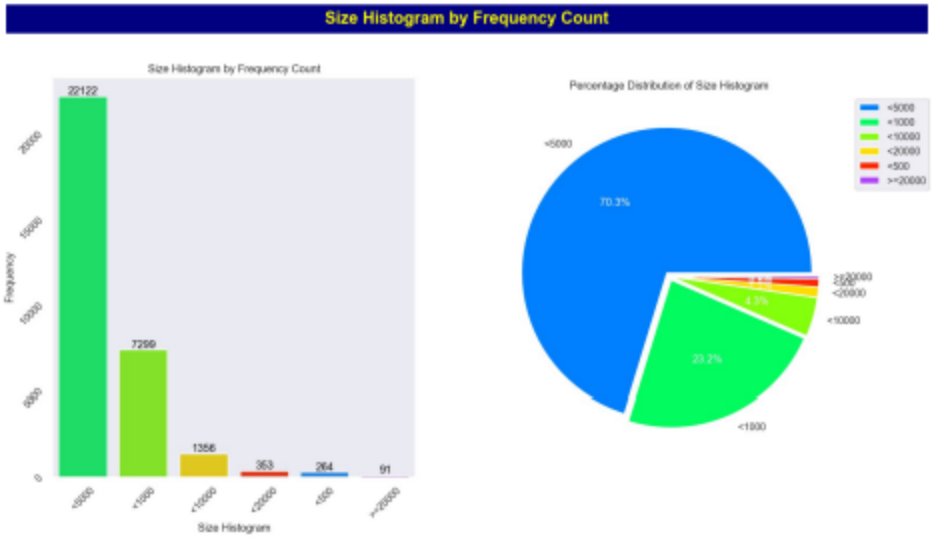




Car Parks by Frequency Count

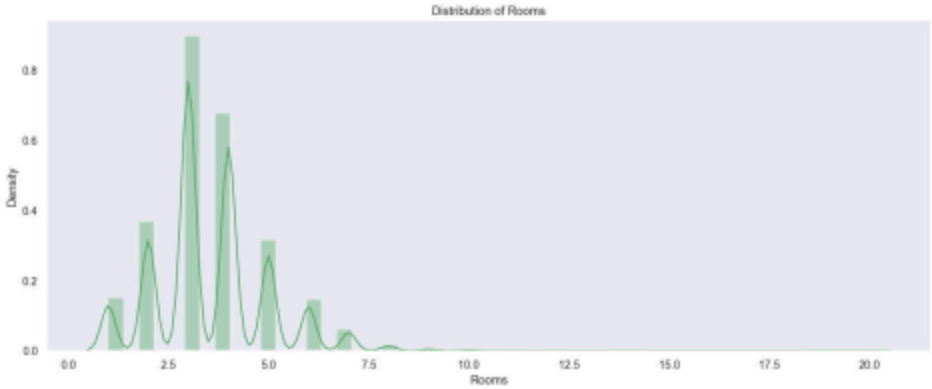




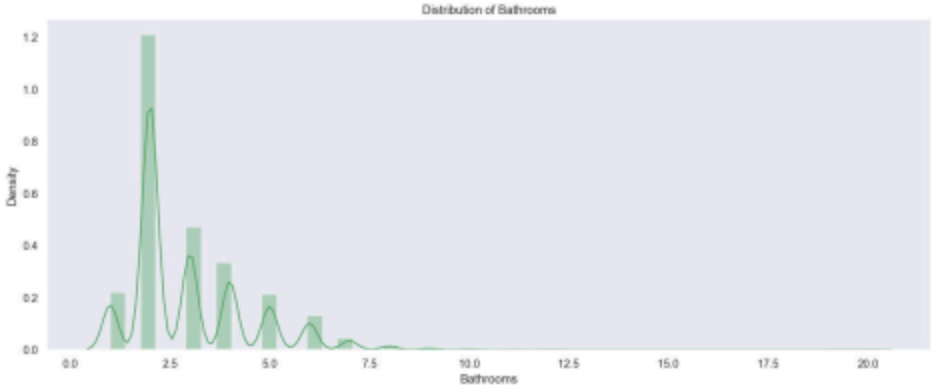



```
# EDA Plot
EDA_COL = ["Rooms", "Bathrooms", "Car Parks"]
EDA_PLT = "gist_ncar"

for idx, col in enumerate(EDA_COL):
    # Plot header
    plot_sub_header("Distribution of {}".format(col))

    plt.figure(figsize=(16, 6), dpi=120)

    sns.distplot(df[col], color="g")
    plt.title("Distribution of {}".format(col))
```

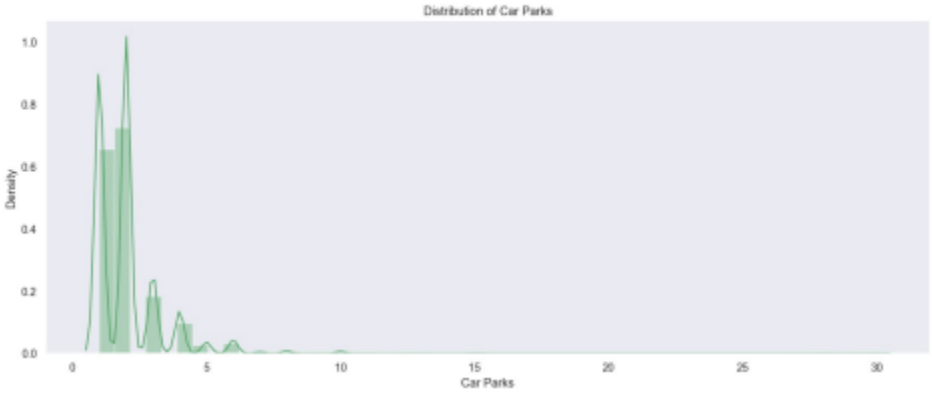
Distribution of Rooms



Distribution of Bathrooms



Distribution of Car Parks

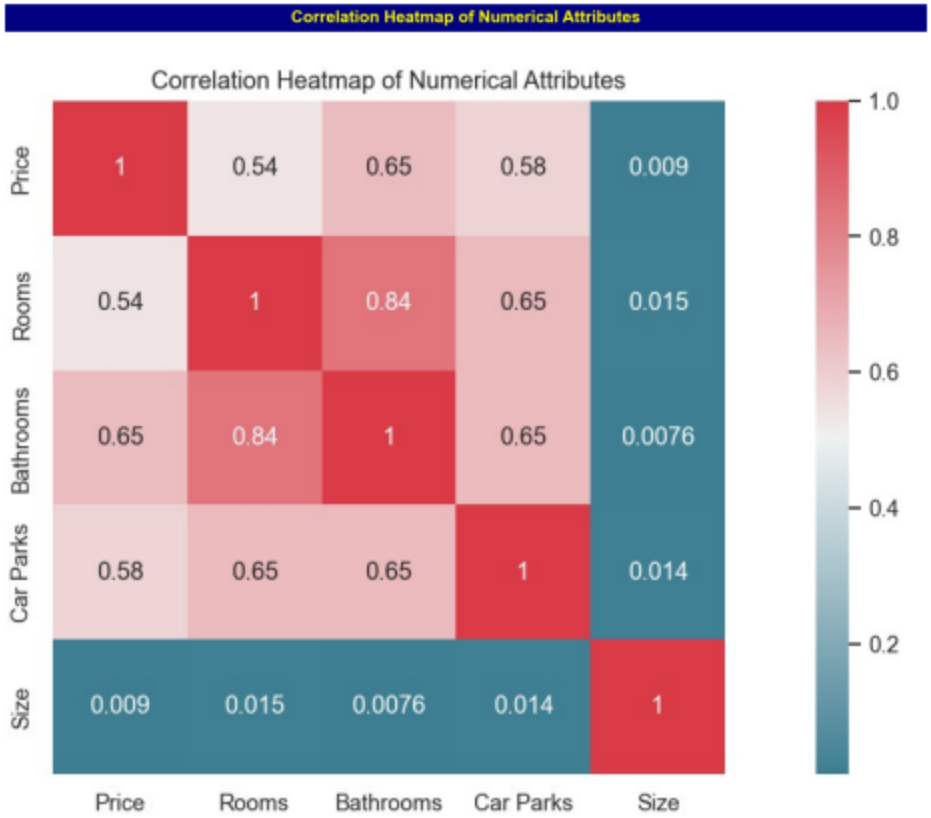


```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 31485 entries, 0 to 53882
Data columns (total 9 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Location        31485 non-null  object
 1   Price           31485 non-null  float64
 2   Rooms           31485 non-null  int64
 3   Bathrooms       31485 non-null  int64
 4   Car Parks       31485 non-null  int64
 5   Property Type   31485 non-null  object
 6   Size            31485 non-null  int64
 7   Furnishing      31485 non-null  object
 8   Size Histogram  31485 non-null  object
dtypes: float64(1), int64(4), object(4)
memory usage: 2.4+ MB
```

```
plot_sub_header("Correlation Heatmap of Numerical Attributes")

plt.figure(figsize=(16, 6), dpi=120)

df_corr = df[["Price", "Rooms", "Bathrooms", "Car Parks", "Size"]]

sns.heatmap(
    df_corr.corr(),
    cmap   = sns.diverging_palette(220, 10, as_cmap=True),
    square = True,
    annot  = True
)
```

```
plt.title("Correlation Heatmap of Numerical Attributes")
plt.show()
```



Dataset Encoding

```
encoder_location      = LabelEncoder()
encoder_property_type = LabelEncoder()
encoder_furnishing    = LabelEncoder()

df["Location"]        =
encoder_location     .fit_transform(df["Location"])
df["Property Type"]   =
encoder_property_type.fit_transform(df["Property Type"])
df["Furnishing"]      =
encoder_furnishing   .fit_transform(df["Furnishing"])
```


Dataset Encoding

```
encoder_location      = LabelEncoder()
encoder_property_type = LabelEncoder()
encoder_furnishing    = LabelEncoder()

df["Location"]        =
encoder_location     .fit_transform(df["Location"])
df["Property Type"]   =
encoder_property_type.fit_transform(df["Property Type"])
df["Furnishing"]      =
encoder_furnishing   .fit_transform(df["Furnishing"])
```

Encoded Dataset

```
       Location      Price  Rooms  Bathrooms  Car Parks  Property Type
Size  \
0            40  1250000.0      3          3          2             16
1335
2            26  1030000.0      3          4          2             12
1875
4            11   900000.0      5          3          2             12
1513
5            74  5350000.0      6          5          4              9
7200
7            74  2600000.0      5          4          4             15
3600
...         ...        ...    ...        ...        ...            ...
...
53862        61   750000.0      3          2          1             12
915
53868        39  1400000.0      4          3          2             12
1544
53869        38   880000.0      1          1          1             12
650
53876        64  2700000.0      6          6          3             12
3973
53882        62   540000.0      4          2          3             12
1313

       Furnishing Size Histogram
0               0          <5000
2               1          <5000
4               1          <5000
5               1         <10000
7               1          <5000
...           ...            ...
53862           1          <1000
53868           0          <5000
```

```
53869           1          <1000
53876           1          <5000
53882           1          <5000

[31485 rows x 9 columns]
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 31485 entries, 0 to 53882
Data columns (total 9 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Location        31485 non-null  int32
 1   Price           31485 non-null  float64
 2   Rooms           31485 non-null  int64
 3   Bathrooms       31485 non-null  int64
 4   Car Parks       31485 non-null  int64
 5   Property Type   31485 non-null  int32
 6   Size            31485 non-null  int64
 7   Furnishing      31485 non-null  int32
 8   Size Histogram  31485 non-null  object
dtypes: float64(1), int32(3), int64(4), object(1)
memory usage: 2.0+ MB
```

```python
# Ensuring the 'Size Histogram' column is numeric
df['Size Histogram'] = pd.to_numeric(df['Size Histogram'],
errors='coerce')

df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 31485 entries, 0 to 53882
Data columns (total 9 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Location        31485 non-null  int32
 1   Price           31485 non-null  float64
 2   Rooms           31485 non-null  int64
 3   Bathrooms       31485 non-null  int64
 4   Car Parks       31485 non-null  int64
 5   Property Type   31485 non-null  int32
 6   Size            31485 non-null  int64
 7   Furnishing      31485 non-null  int32
 8   Size Histogram  0 non-null      float64
dtypes: float64(2), int32(3), int64(4)
memory usage: 2.0 MB
```

Target Variable Processing

```python
# Drop the unwanted attributes from X and allocate target variables to
Y
x = df.drop(["Price", "Size Histogram"], axis=1)
y = df["Price"]
```

Recursive Feature Elimination (RFE)

```python
# Plot header
plot_sub_header("RFE-Feature Elimination")

# Define the RFE using DCT with specified features to keep
rfe = RFE(
    estimator            = DecisionTreeClassifier(),
    n_features_to_select = 5,
    step                 = 1,
    verbose              = 1,
    importance_getter    = "auto"
)

# RFE model fitting
rfe.fit(x, y)

# Update dataset to RFE selected attributes
x = x[x.columns[rfe.support_]]
```

RFE-Feature Elimination

```
Fitting estimator with 7 features.
Fitting estimator with 6 features.

# Plot header
plot_sub_header("RFE-Reduced Attributes")

display(x)
```

RFE-Reduced Attributes

```
       Location  Bathrooms  Car Parks  Size  Furnishing
0            40          3          2  1335           0
2            26          4          2  1875           1
4            11          3          2  1513           1
5            74          5          4  7200           1
7            74          4          4  3600           1
...         ...        ...        ...   ...         ...
53862        61          2          1   915           1
53868        39          3          2  1544           0
```

```
53869        38          1          1   650           1
53876        64          6          3  3973           1
53882        62          2          3  1313           1

[31485 rows x 5 columns]
```

Dataset Splitting

```
# Split the dataset into training and testing portions
x_train, x_test, y_train, y_test = train_test_split(
    x,
    y,
    test_size    = TEST_SIZE,
    random_state = RANDOM_STATE
)
```

Ranfom Forest Regressor - Model Definition and Model Building

```
rfr = RandomForestRegressor(
    n_estimators                = 100,
    criterion                   = "squared_error",
    max_depth                   = None,
    min_samples_split           = 2,
    min_samples_leaf            = 1,
    min_weight_fraction_leaf    = 0.0,
    max_features                = 1.0,
    max_leaf_nodes              = None,
    min_impurity_decrease       = 0.0,
    bootstrap                   = True,
    oob_score                   = False,
    n_jobs                      = -1,
    random_state                = RANDOM_STATE,
    verbose                     = 0,
    warm_start                  = False,
    ccp_alpha                   = 0.0,
    max_samples                 = None
)

rfr.fit(x_train, y_train)
```

```
RandomForestRegressor(n_jobs=-1, random_state=331)
```

Ranfom Forest Regressor - Model Performance Evaluation

```
rfr_y_pred = rfr.predict(x_test)
rfr_mae    = mean_absolute_error(y_test, rfr_y_pred)
rfr_rmse   = np.sqrt(mean_squared_error(y_test, rfr_y_pred))
rfr_mape   = mean_absolute_percentage_error(y_test, rfr_y_pred)
rfr_r2s    = r2_score(y_test, rfr_y_pred)
```

```
print("Mean Absolute Error            (MAE ):", rfr_mae)
print("Root Mean Squared Error        (RMSE):", rfr_rmse)
print("Mean Absolute Percentage Error (MAPE):", rfr_mape)
print("R-Squared Score                (R2  ):", rfr_r2s)
```

```
Mean Absolute Error            (MAE ): 275741.9382954099
Root Mean Squared Error        (RMSE): 813558.681872169
Mean Absolute Percentage Error (MAPE): 2.7367853375440188
R-Squared Score                (R2  ): 0.8512289776513552
```

Gradient Boosting Regressor - Model Definition and Model Building

```
gbr = GradientBoostingRegressor(
    loss                          = "squared_error",
    learning_rate                 = 0.1,
    n_estimators                  = 100,
    subsample                     = 1.0,
    criterion                     = "friedman_mse",
    min_samples_split             = 2,
    min_samples_leaf              = 1,
    min_weight_fraction_leaf      = 0.0,
    max_depth                     = 3,
    min_impurity_decrease         = 0.0,
    init                          = None,
    random_state                  = RANDOM_STATE,
    max_features                  = None,
    alpha                         = 0.9,
    verbose                       = 0,
    max_leaf_nodes                = None,
    warm_start                    = False,
    validation_fraction           = 0.2,
    n_iter_no_change              = None,
    tol                           = 1e-4,
    ccp_alpha                     = 0.0
)

gbr.fit(x_train, y_train)
```

```
GradientBoostingRegressor(random_state=331, validation_fraction=0.2)
```

Gradient Boosting Regressor - Model Performance Evaluation

```
gbr_y_pred = gbr.predict(x_test)
gbr_mae    = mean_absolute_error(y_test, gbr_y_pred)
gbr_rmse   = np.sqrt(mean_squared_error(y_test, gbr_y_pred))
gbr_mape   = mean_absolute_percentage_error(y_test, gbr_y_pred)
gbr_r2s    = r2_score(y_test, gbr_y_pred)

print("Mean Absolute Error            (MAE ):", gbr_mae)
print("Root Mean Squared Error        (RMSE):", gbr_rmse)
```

```
print("Mean Absolute Percentage Error (MAPE):", gbr_mape)
print("R-Squared Score                (R2  ):", gbr_r2s)
```

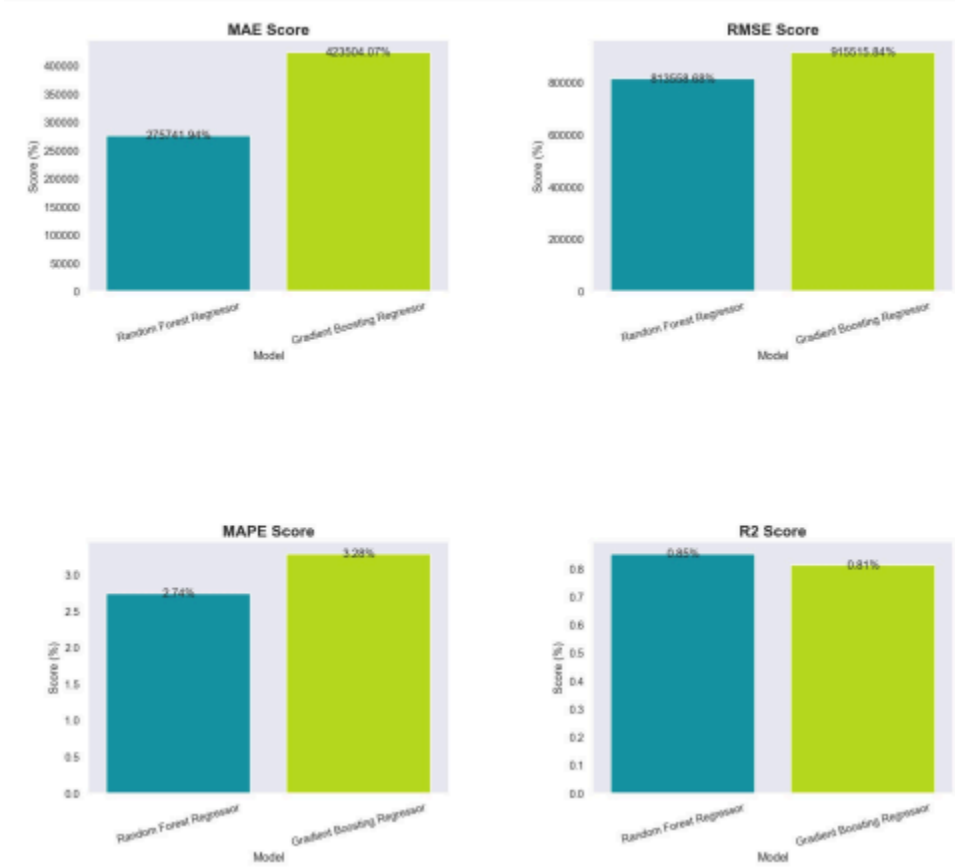
```
Mean Absolute Error            (MAE ): 423504.07029773865
Root Mean Squared Error        (RMSE): 915515.8378504089
Mean Absolute Percentage Error (MAPE): 3.28016467013593
R-Squared Score                (R2  ): 0.8116037287665433
```

Machine Learning Model Performance Comparison

```
MODEL_NAME_L = ["Random Forest Regressor", "Gradient Boosting
Regressor"]
perf_mae     = [rfr_mae , gbr_mae ]
perf_rmse    = [rfr_rmse, gbr_rmse]
perf_mape    = [rfr_mape, gbr_mape]
perf_r2s     = [rfr_r2s , gbr_r2s ]

# Build Machine Learning model performance score to a DataFrame
perf = pd.DataFrame(
    {
        "Model": MODEL_NAME_L,
        "MAE"  : perf_mae,
        "RMSE" : perf_rmse,
        "MAPE" : perf_mape,
        "R2S"  : perf_r2s,
    }
)

MODEL_LIST = [
    rfr, gbr
]

# Machine Learning model performance visualization

# Plot header
plot_sub_header("Model Performance Visualization")

fig, axes = plt.subplots(2, 2, figsize=(16, 14), dpi=120)
fig.subplots_adjust(wspace=0.40, hspace=1.00)

EDA_PLT   = "nipy_spectral"

# Plot MAE
sns.barplot(perf, ax=axes[0, 0], x="Model", y="MAE", palette=EDA_PLT)
axes[0][0].set_title("MAE Score", fontsize=16, weight="bold")
axes[0][0].tick_params(axis="x", rotation=15)
axes[0][0].set(ylabel="Score (%)")
axes[0][0].set_xticklabels(MODEL_NAME_L)
plot_add_text(axis=axes[0][0], x=perf["Model"], y=perf["MAE"])
```

```
# Plot RMSE
sns.barplot(perf, ax=axes[0, 1], x="Model", y="RMSE", palette=EDA_PLT)
axes[0][1].set_title("RMSE Score", fontsize=16, weight="bold")
axes[0][1].tick_params(axis="x", rotation=15)
axes[0][1].set(ylabel="Score (%)")
axes[0][1].set_xticklabels(MODEL_NAME_L)
axes[0][1].ticklabel_format(style="plain", axis="y")
plot_add_text(axis=axes[0][1], x=perf["Model"], y=perf["RMSE"])

# Plot MAPE
sns.barplot(perf, ax=axes[1, 0], x="Model", y="MAPE", palette=EDA_PLT)
axes[1][0].set_title("MAPE Score", fontsize=16, weight="bold")
axes[1][0].tick_params(axis="x", rotation=15)
axes[1][0].set(ylabel="Score (%)")
axes[1][0].set_xticklabels(MODEL_NAME_L)
plot_add_text(axis=axes[1][0], x=perf["Model"], y=perf["MAPE"])

# Plot R2S
sns.barplot(perf, ax=axes[1, 1], x="Model", y="R2S", palette=EDA_PLT)
axes[1][1].set_title("R2 Score", fontsize=16, weight="bold")
axes[1][1].tick_params(axis="x", rotation=15)
axes[1][1].set(ylabel="Score (%)")
axes[1][1].set_xticklabels(MODEL_NAME_L)
plot_add_text(axis=axes[1][1], x=perf["Model"], y=perf["R2S"])

plt.show()
```

Model Performance Visualization



Model Testing and Deployment

```
def house_price_predict(MY_DATA):
    encoder_location      =
pickle.load(open("./encoder/encoder_location.pkl", "rb"))
    encoder_property_type =
pickle.load(open("./encoder/encoder_property_type.pkl", "rb"))
    encoder_furnishing    =
pickle.load(open("./encoder/encoder_furnishing.pkl", "rb"))

    my_data               = pd.DataFrame([MY_DATA])
    my_data[0]            = encoder_location  .transform(my_data[0])
    my_data[4]            = encoder_furnishing.transform(my_data[4])
    my_rfr_y_pred         = rfr.predict(my_data)
    my_gbr_y_pred         = gbr.predict(my_data)
```

```
    # Plot header
    plot_sub_header("House Price Prediction")

    print("Random Forest Regressor    : RM ",
round(float(str(my_rfr_y_pred)[1:-1]), 2))
    print("Gradient Boosting Regressor: RM ",
round(float(str(my_gbr_y_pred)[1:-1]), 2))

house_price_predict(
    [
        "KLCC, Kuala Lumpur",       # Location
        3,                          # Bathrooms
        2,                          # Car Parks
        1335,                       # Size
        "Fully Furnished"           # Furnishing
    ]
)
```

House Price Prediction

```
Random Forest Regressor    : RM  1397426.79
Gradient Boosting Regressor: RM  1734515.98

house_price_predict(
    [
        "Bukit Jalil, Kuala Lumpur",# Location
        3,                          # Bathrooms
        2,                          # Car Parks
        791,                        # Size
        "Partly Furnished"          # Furnishing
    ]
)
```

House Price Prediction

```
Random Forest Regressor    : RM  647678.9
Gradient Boosting Regressor: RM  540140.63

house_price_predict(
    [
        "Mont Kiara, Kuala Lumpur", # Location
        5,                          # Bathrooms
        3,                          # Car Parks
        3041,                       # Size
        "Partly Furnished"          # Furnishing
    ]
)
```

House Price Prediction

```
Random Forest Regressor    : RM  2512626.19
Gradient Boosting Regressor: RM  2495446.26

# EOF
```